Exercise 03: Convolutional Neural Network - Step-by-step guide

Submission Requirements

- Create `cnn.ipynb`
- Export it to `cnn.html`
- Zip both files into `ex03.zip`
- Submit on MaMPF before June 11, 2026, 21:00.

Part 1: Run the Intro PyTorch Network (5 Points)

Step 1: Set Up Environment

Install PyTorch.

Using conda:

conda create -n mle python=3.12
conda activate mle

conda install pytorch torchvision torchaudio -c pytorch
pip install matplotlib numpy notebook

Step 2: Create Notebook

Create:

cnn.ipynb

Copy the entire `intro.py` code from the exercise into separate notebook cells.

Suggested structure:

Cell 1

Imports

Cell 2

Load MNIST

Cell 3

Helper functions

init_weights()
rectify()
RMSprop()

Cell 4

Model

model()

Cell 5

Initialize weights

Cell 6

Training loop

Cell 7

Plot losses

Step 3: Run Training

Verify:

- Training loss decreases
- Test loss decreases
- Plot appears correctly

Take screenshots for your report.

Part 2: Dropout (8 Points)

Concept

Dropout randomly "turns off" neurons during training.

Example:

Without dropout:

Input → Neuron A → Output

With dropout:

Input → X (disabled)

This prevents neurons from becoming overly dependent on each other.

Step 1: Implement Dropout

Create:

```
def dropout(X, p_drop=0.5):

    if p_drop <= 0 or p_drop >= 1:
        return X

    mask = torch.bernoulli(
        torch.ones_like(X) * (1 - p_drop)
    )
```

```
    return X * mask / (1 - p_drop)
```

The scaling

/(1-p_drop)

keeps expected activations unchanged.

Step 2: Build New Model

```
def dropout_model(
        X,
        w_h,
        w_h2,
        w_o,
        p_drop_input,
        p_drop_hidden):

    X = dropout(X, p_drop_input)

    h = rectify(X @ w_h)

    h = dropout(h, p_drop_hidden)

    h2 = rectify(h @ w_h2)

    h2 = dropout(h2, p_drop_hidden)

    return h2 @ w_o
```

Notice:

Dropout applied:

1. Input layer
2. First hidden layer
3. Second hidden layer

Exactly as requested.

Step 3: Training Mode

Use:

```
dropout_model(
    x,
    w_h,
    w_h2,
    w_o,
    0.2,
    0.5
)
```

Step 4: Test Mode

During testing:

```
dropout_model(
    x,
    w_h,
    w_h2,
    w_o,
    0.0,
    0.0
)
```

Why?

Because during evaluation we want to use the entire network.

If neurons are still randomly dropped, predictions become unstable.

This is one question the sheet explicitly asks.

Step 5: Report

Write 3–5 sentences:

- Dropout randomly disables neurons.
- Prevents co-adaptation.
- Forces robust features.
- Reduces overfitting.
- Usually improves test error.

Then compare:

Model	**Test Error**
Baseline	?
Dropout	?

Part 3: Parametric ReLU (10 Points)

Concept

Regular ReLU:

$x > 0 \rightarrow x$
$x \leq 0 \rightarrow 0$

PReLU:

$x > 0 \rightarrow x$
$x \leq 0 \rightarrow a*x$

where a is learnable.

The exercise defines the following:

x if x > 0
a*x if x <= 0

Step 1: Define PReLU

```
def PRelu(X, a):
    return torch.where(X > 0, X, a * X)
```

Step 2: Initialize Learnable Parameters

For first hidden layer:

```
a1 = torch.ones(625) * 0.25
a1.requires_grad = True
```

For the second hidden layer:

```
a2 = torch.ones(625) * 0.25
a2.requires_grad = True
```

Step 3: Create Model

```
def prelu_model(
        X,
        w_h,
        w_h2,
        w_o,
        a1,
        a2):

    h = PRelu(X @ w_h, a1)

    h2 = PRelu(h @ w_h2, a2)

    return h2 @ w_o
```

Step 4: Add Parameters to Optimizer

```
optimizer = RMSprop(
    params=[
```

```
        w_h,
        w_h2,
        w_o,
        a1,
        a2
    ]
)
```

The exercise specifically requires this.

Step 5: Compare Results

Create table:

Model	Test Error
Baseline	?
Dropout	?
PReLU	?

Part 4: Convolutional Neural Network (17 Points)

This is the biggest section.

Step 1: Reshape Images

Instead of:

x.reshape(batch_size, 784)

Use:

x = x.reshape(-1, 1, 28, 28)

Required by exercise.

Step 2: Create Convolution Weights

Conv1

w_c1 = init_weights((32, 1, 5, 5))

Conv2

w_c2 = init_weights((64, 32, 5, 5))

Conv3

w_c3 = init_weights((128, 64, 3, 3))

These dimensions come directly from the exercise.

Step 3: Calculate Feature Map Sizes

Input

1 × 28 × 28

Conv1 (5×5)

28 - 5 + 1 = 24

Output:

32 × 24 × 24

Pool1

32 × 12 × 12

Conv2

12 - 5 + 1 = 8

Output:

64 × 8 × 8

Pool2

64 × 4 × 4

Conv3

4 - 3 + 1 = 2

Output:

128 × 2 × 2

Pool3

128 × 1 × 1

Step 4: Flatten

h = torch.reshape(h, (h.shape[0], -1))

Result:

128 features

because:

128 × 1 × 1 = 128

Step 5: Fully Connected Layer

w_h2 = init_weights((128, 625))

This is the "number_of_output_pixel" requested in the assignment.

Step 6: Output Layer

w_o = init_weights((625, 10))

Step 7: CNN Model

Structure:

conv1
→ relu
→ maxpool
→ dropout

conv2
→ relu
→ maxpool
→ dropout

conv3
→ relu
→ maxpool
→ dropout

flatten

fully connected

output

Visualization Task

Plot:

Original image

plt.imshow(...)

Feature maps

Choose three filters:

feature_map[0,0]
feature_map[0,1]
feature_map[0,2]

Plot them.

Filter weights

Plot:

w_c1[0,0]
w_c1[1,0]
w_c1[2,0]

These are the learned 5×5 kernels.

Final Experiment (Choose One)

Easiest option:

Remove one convolution layer

or

Add one convolution layer

Much easier than image augmentation.

Example:

Conv1
Conv2
FC
Output

Compare test error.

Final Report Table

The exercise strongly suggests an overview table.

Create:

Model	Test Error
Fully Connected	
Dropout	
PReLU	
CNN	
Modified CNN	

Recommended Work Order

1. Run baseline network (Part 1)
2. Implement dropout (Part 2)
3. Implement PReLU (Part 3)
4. Implement CNN (Part 4)
5. Produce plots
6. Run extra experiment
7. Create comparison table
8. Export notebook → HTML
9. Zip and submit

This order minimizes debugging because each section builds on the previous one.